

Weather Predictor — Tutorial de progreso

Autor: José Rubio · **Fecha:** 2026-06-21 · **Versión:** rev 5 (Pi + Kalshi + AI agent)

1. Resumen ejecutivo

Weather Predictor es una aplicación educativa que predice la temperatura máxima diaria y la compara contra los precios del mercado de contratos de eventos de **Kalshi** (KXHIGHT* / KXHIGHT*) para esa misma temperatura. El scope curado son siete estaciones — **KPHX (Phoenix)**, **KLAX (Los Angeles)**, **KLAS (Las Vegas)**, **KLGA (New York)**, **KBOS (Boston)**, **KMIA (Miami)** y **KMDW (Chicago)** — elegidas por liquidez en Kalshi y diversidad climática (3 desierto + 2 costa este + Miami subtropical + Chicago continental).

El objetivo no es apostar con dinero real, sino medir cuánto "edge" tiene un modelo ensemble propio frente al mercado y cómo mejora esa precisión con el paso de las horas del día.

El sistema corre 24/7 en una **Raspberry Pi 4B (8GB)** accesible vía Tailscale en `100.83.162.24`. Tres servicios en puertos separados:

- `:8000` — Weather predictor (este proyecto, 7 estaciones)
- `:8001` — Crypto predictor (proyecto paralelo BTC quarterly, mismo enfoque)
- `:8080` — Dashboard agregado (escanea 20 estaciones, monitor BTC, tab AI)

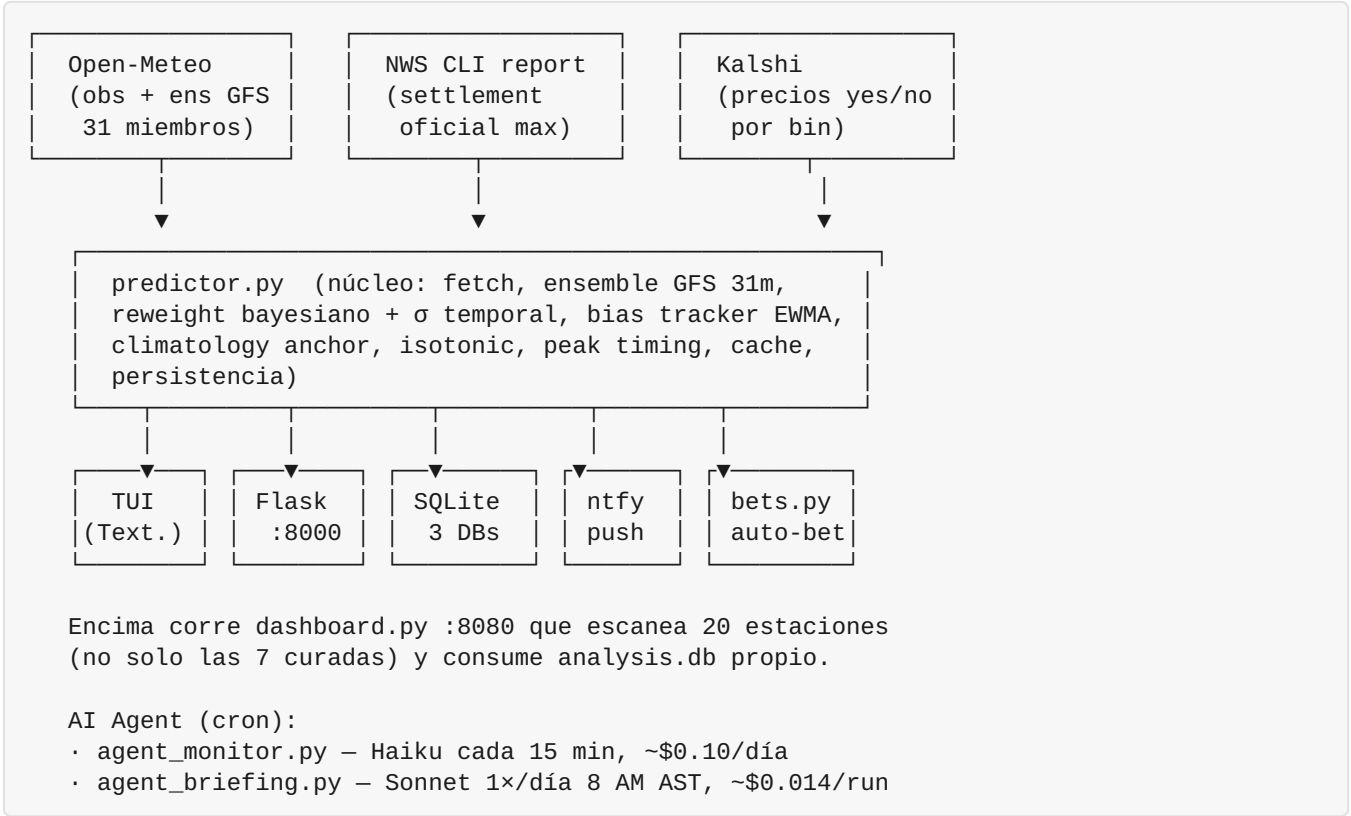
Toda la información se persiste en SQLite (5 bases entre proyectos) para análisis retrospectivo. Polling adaptativo cada 3 min en pico, 10 min fuera.

2. Objetivo del proyecto

- **Predecir** el máximo diario en °F con una distribución de probabilidad, no un solo número.
- **Comparar** esa distribución contra los precios de Kalshi para las 7 estaciones curadas.
- **Medir** si el modelo es consistente, calibrado, y si ofrece ventaja sistemática frente al mercado.
- **Identificar** días "difíciles" (fronts, anomalías, ruptura de régimen) donde lo seguro es saltar la apuesta.
- **Aprender** sobre ensemble forecasting, calibración probabilística y prediction markets sin usar dinero real.

El diseño apunta a uso diario: el iPad muestra el dashboard `:8080` vía Tailscale por la mañana, recibo el **briefing matutino** automático del agente AI a las 8:00 AM AST y decido qué estación tiene la mejor combinación de edge + dificultad baja.

3. Arquitectura



- **Polling adaptativo:** un thread de background refresca obs + ensemble cada **3 min en ventana de pico** (12–17h local según estación) y **10 min fuera**. Durante pico invalida cache de obs para leer METAR fresco.
- **Persistencia:**
 - market_cache.db — precios mercado + nuestro p por bin (Kalshi snapshots)
 - calibration.db — prediction_snapshots, day_outcomes, day_summary, simulated_bets, bias_tracker, isotonic_state
 - climatology.db — 30 años de max diarios (Open-Meteo archive cacheado)
 - analysis.db — dashboard :8080, 20 estaciones
 - agent.db — decisiones AI, budget, briefings
- **Acceso remoto:** los tres servicios visibles desde iPad/laptop vía Tailscale (100.83.162.24) o LAN local.
- **Auto-start:** ~/start_all.sh arranca los 3 servicios al rebooteo (cron @reboot).

4. Fuentes de datos

Fuente	Uso	Formato
Open-Meteo Current	Temperatura actual, dewpoint, humedad, viento, presión	API JSON, llamada cada 3/10 min
Open-Meteo Ensemble GFS	31 miembros del GFS, hora a hoy/mañana/ pasado	API JSON: temperatura + precipitación + nieve

Fuente	Uso	Formato
Open-Meteo Archive	30 años de max diarios por estación	API JSON, cache SQLite local
NWS Climate Report (CLI)	Max oficial del día (settle source)	Scraping texto NWS
Kalshi REST API	Precios yes/no por bin de temperatura	JSON oficial (sin auth, endpoints públicos)
Modelos externos	ECMWF/GFS/NAM/HRRR/Wunderground como sanity check	Open-Meteo seasonal API

Settle: desde el swap a Kalshi, **NWS CLI es la única fuente de settlement** (sin fallback a Open-Meteo archive). Los días pendientes esperan a que el reporte final NWS llegue. NY (KXHIGHNY) liquida con KNYC (Central Park), no LGA — KLGA sigue siendo nuestro id de predicción METAR pero la prob se ajusta al settle de Central Park.

5. Modelo predictivo

La idea central: en vez de predecir un solo número, generar **una distribución de probabilidad** sobre cuál va a ser el máximo del día.

5.1 Ensemble GFS (baseline)

Open-Meteo ofrece los 31 miembros del ensemble GFS por separado. Cada miembro es una simulación con pequeñas perturbaciones en condiciones iniciales. Para cada miembro extraemos el max del día, dando 31 muestras de la distribución predictiva. Calculamos p10, mediana, p90 y modal directamente.

5.2 Reweight bayesiano + σ temporal

A medida que avanza el día, ya conocemos la temperatura real las primeras horas. Esa observación reweightea los miembros del ensemble: los miembros que "se parecen" a lo observado reciben más peso; los que divergieron mucho, menos.

Implementación: softmax sobre la suma de errores cuadrados (SSE) estandarizada por un **σ dependiente de la hora:**

- Pico (12-17h): $\sigma=1.5^{\circ}\text{F}$ (más exigente, más discriminación)
- Adyacentes: $\sigma=2.0^{\circ}\text{F}$
- Medias: $\sigma=2.5^{\circ}\text{F}$
- Lejanas: $\sigma=3.5^{\circ}\text{F}$

Re-muestreamos a $N=500$. El reporte muestra "eff N" (miembros efectivos tras reweight). Si $\text{eff_N} < 20 \rightarrow$ "reweight colapsado", la predicción es inestable.

5.2.1 Detector de ruptura de régimen

Si la observación de una hora cae fuera de la envolvente **p1–p99** del ensemble (con margen 4°F para absorber sesgos de estaciones urbanas), esa hora se marca como **ruptura**. Con ≥ 2 horas rotas:

- Se fuerza la dificultad del día a 100 ("ruptura de régimen")
- Push ntfy al iPad

- `/cross` recomienda saltar el día
- El agente AI (monitor + briefing) filtra todas las opps de esa estación

Lección importante (2026-06-21): la "triple convergencia" (nuestro modelo + mediana de externos + mispricing Kalshi) **NO funciona** con régimen roto o `difficulty=100`. Los modelos externos también pierden calibración en regímenes extremos — no son independientes del nuestro. Regla dura: nunca recomendar entry si `difficulty>70`, `régimen roto` o `eff_N<25`.

5.3 Peak timing

Además del valor del máximo, predecimos **cuándo**. Para cada miembro extraemos la hora del max y reportamos modal, p10/p50/p90, P(ya ocurrió), P(ocurrirá en próximas 1/2/3/6 horas).

5.4 Bias tracker EWMA

Por estación mantenemos un EWMA del bias del modelo (predicción modal vs observación). Si la estación tiene bias persistente +2°F (modelo sobrepronostica) → la mediana se ajusta hacia abajo. Activo desde 2026-04-29.

5.5 Climatology anchor

En heatwave (`ext_diff ≥ +1.5°F` en oeste/sur o `p≥85%` de cola alta), anclamos a la mediana de modelos externos en vez de NO-sellar bins altos. Memoria de fallos previos confirmó que en heatwave los externos ganan al modelo crudo.

5.6 Calibración isotónica

PAV (Pool-Adjacent-Violators) sobre snapshots históricos (`predicted_p`, `outcome`). Gate actual: `≥20` pares y `≥7` días distintos. Cuando pasa el gate, se aplica en vivo a ladder/comparison. Estado actual: variable según estación.

6. Integración con el mercado (Kalshi)

Desde el swap de 2026-05-08 usamos Kalshi como mercado en lugar de Robinhood. Kalshi tiene API REST oficial (sin auth para market data), bins definidos por rango de temperatura (`[lo, hi]` , con tails `≤X` y `≥Y`), y settlement alineado con NWS CLI — exactamente lo que necesitamos para no tener divergencia de fuentes.

Series tracker: `KXHIGHTPHX` (Phoenix), `KXHIGHTLAX` (LA), `KXHIGHTLV` (Las Vegas), `KXHIGHTNY` (NY/Central Park), `KXHIGHTBOS` (Boston), más Miami y Chicago.

6.1 `/comparison` — Kalshi vs modelo

Tabla bin por bin con `yes_mid` de Kalshi y nuestra probabilidad post-calibración (bayes + bias + isotonic). Barra visual y diff en pp. Esta es la **predicción final** del modelo, la que usa el bot para auto-bets — no confundir con el `/analysis` del dashboard `:8080` que muestra el ensemble crudo.

6.2 /ladder — Decision ladder

Para cada threshold ($\pm N$ alrededor de mediana):

- `yes/no` nuestro vs `yes/no` Kalshi
- `edge` en puntos porcentuales
- EV al apostar \$1: $EV_{yes} = (p-k)/k$, $EV_{no} = (k-p)/(1-k)$
- Kelly fraction: $f^*_{yes} = (p-k)/(1-k)$, $f^*_{no} = (k-p)/k$
- Pill **YES/NO** señalando lado recomendado

6.3 /edge y /movement

- **Edge tracking** — contratos con mayor edge actual + análisis histórico (edges por bucket, ROI hipotético).
- **Movement** — evolución temporal de `yes_mid` Kalshi vs `our_p` para un bin dado.

6.4 /bets — Simulador P&L

Cuando `|edge| ≥ 5pp`, el sistema "apuesta" \$10 hipotéticos en el lado correcto (guard contra precios ≤ 0.01 o ≥ 0.99). Al settlearse el día, calcula payoff real. Filtros anti-pérdidas vigentes (desde 2026-05-26):

- Bloqueo por bias o losing streak
- Mid-bin via `our_pred` (no apostar bins muy estrechos)
- Divergence overnight con externos
- Auto-cleanup de bets erróneas

Dedupe por `(station, date, ticker)`.

6.5 Tail-negation preferida

Con modelo sesgado, **preferir NO $[\leq X]$ de cola con edge**

≥40pp sobre point-bets centrales (validado KLGA 2026-05-25: ganó +\$7.50). Memoria activa.

6.6 /calibration + /history

- **Calibration**: reliability diagram dual (nuestro vs Kalshi) + Brier per-bin.
- **History**: tabla diaria con max observado, Brier nuestro, Brier Kalshi, mejor edge, si fue correcto. Total días ganando al mercado.

6.7 Push notifications (ntfy)

Opt-in vía env `NTFY_TOPIC`. Edge $\geq 10pp$ → push al iPad. Settle → push con Briers comparados. El briefing matutino del AI agent también puede empujar a ntfy si está configurado.

7. Guía por página

La web `predictor.py` corre en `http://100.83.162.24:8000`. El dashboard agregado en `:8080`. Crypto en `:8001`.

Para decidir si apostar hoy

- `/` (**dashboard principal :8000**) — Todo del día en una pantalla: temp actual, pronóstico, distribución del ensemble, **badge de dificultad**, **dropdown KPHX/KLAX/KLAS/KLGA/KBOS/KMIA/KMDW** sin reiniciar, cards de clima/viento/presión, peak timing, precipitación, aserciones personalizadas. Línea "edge máx" arriba indica la decisión final del bot.
- `/cross` — Las estaciones lado a lado con **ranking** y **pill de recomendación**: "apuesta a STATION SIDE +Xpp" (#1 con edge $\geq 5pp$ y dificultad < 30) o "⚠ mejor saltar hoy". Selector hoy/mañana/pasado.
- `/comparison` — Barra visual bin-por-bin: yes_mid Kalshi vs nuestra prob **post-calibración** + diff en pp.
- `/ladder` — Por temperatura umbral: nuestro yes/no vs Kalshi, edge en pp, EV%, Kelly, pill YES/NO. Selector ventana $\pm 2/\pm 3/\pm 4/\pm 6/\pm 10$ /todo.
- `/edge` — Tabla edges disponibles + performance histórica por bucket.

Dashboard agregado :8080

- `/analysis` — 20 estaciones (más allá de las 7 curadas), bins Kalshi vs `our_p` raw, tabla de aseveraciones del user con probabilidad declarada. `our_p` aquí es **ensemble crudo (count_in_bin/N)** — no aplica bayes/bias/isotonic. Para predicción final ir a `:8000/comparison`.
- `/btc-quarter` — Monitor BTC quarterly (proyecto paralelo, ver crypto predictor).
- `/ai` — Tab del AI agent: budget, gasto del día, briefing matutino destacado arriba, tabla de decisiones del monitor.

Para entender el clima

- `/timing` — Hora del pico: modal, rango p10-p90, P(ya ocurrió), P(en próximas 1/2/3/6h).
- `/precip` — Probabilidad lluvia/nieve hoy/mañana/pasado.

Para ver si el sistema funciona

- `/reweight` — Diagnóstico hora por hora del reweight: obs vs ensemble p10/p50/p90, σ aplicado (verde=pico), n miembros matched, flag dentro/fuera p1–p99. Rupturas en rojo.
- `/calibration` — Reliability dual + Brier crudo vs calibrado.
- `/history` — Tabla diaria, Brier nuestro vs Kalshi, mejor edge, hit/miss. Total días ganando.
- `/bets` — Simulador P&L: n bets, win rate, stake, payoff, ROI.
- `/movement` — Evolución temporal Kalshi vs nuestro pronóstico para un bin.

Utilidades

- `/notify`, `/export` (CSV de 5 tablas), `/status` (salud), `/about` (este tutorial).

8. AI Agent

Componente añadido en 2026-06-21. Dos procesos cron-driven que usan la API de Anthropic Claude:

8.1 agent_monitor.py (cada 15 min, Haiku 4.5)

Lee `analysis.db` (snapshots + bins de las 20 estaciones del dashboard) + aseveraciones del usuario, manda contexto estructurado a Claude Haiku con un system prompt que codifica:

- Reglas duras: spread $\leq 5^\circ\text{F}$, KLGA $\rightarrow$ KNYC, conviction tiers, lógica de side correcta
- Detección de mercado settled (ens_spread=0 + obs $\approx$ ens_med)
- Triple convergencia (modelo + ext_med + mispricing) solo con difficulty ≤ 70 + eff_N ≥ 25 + ext_spread $\leq 4^\circ\text{F}$

Output JSON con oportunidades (conviction high/med). Guarda en `agent.db`. Costo medido: $\sim \$0.001/\text{call}$ $\rightarrow$ $\sim \$0.10/\text{día}$. Budget cap soft \$14.50; auto-pausa al alcanzarlo.

8.2 agent_briefing.py (8:00 AM AST, Sonnet 4.6)

Lee snapshots actuales + outcomes de ayer + última decisión del monitor. Genera briefing narrativo de 6-8 líneas con setups del día, estaciones a evitar, lección de ayer. Se muestra al tope de `/ai`. Push ntfy opcional.

Costo medido: $\sim \$0.014/\text{call}$ $\rightarrow$ $\sim \$0.42/\text{mes}$.

8.3 Control y visibilidad

- Tab `/ai` en dashboard: budget cap, gasto total, gasto hoy, proyección mensual, estado (activo/pausado con botón toggle), briefing destacado, tabla de últimos 20 ciclos con opps detectadas y razonamiento.
- Hard cap definible en Anthropic Console por si el soft cap falla.
- Lecciones aprendidas se guardan como memorias del usuario (`feedback_*.md`) y se inyectan en el system prompt de futuras corridas.

9. Decisiones abiertas

1. **Más estaciones curadas.** Hoy 7. Kalshi tiene ~ 12 con liquidez decente. ¿Expandir o mantener foco?
2. **Cross-station Bayesian reweight.** Usar obs matutinas de KEWR/KJFK para reweightear KLGA. No implementado.
3. **AI agent fase 2 — chat endpoint `/ask`.** Conversación con el modelo sobre estado actual (usaría Sonnet, $\sim \$0.03/\text{pregunta}$).
4. **AI agent fase 3 — post-mortem automático.** Al settlearse el día, agente compara predicción vs outcome y guarda lecciones en `agent_lessons.db` que se inyectan al prompt del día siguiente.
5. **Más features ensemble.** Ya: lluvia/nieve. Falta: viento max, nubosidad, ráfagas.
6. **Gate isotónica.** Hoy ≥ 20 pares, ≥ 7 días. ¿Bajar a 5 días para activar antes?

10. Próximos pasos por ROI

Completados desde rev 3 (abril $\rightarrow$ junio 2026):

- ✓ Swap Robinhood $\rightarrow$ Kalshi (2026-05-08) con KLGA override a Central Park

- ✓ Bias tracker EWMA por estación
- ✓ Climatology anchor en heatwave
- ✓ Detector de divergencia con modelos externos + card en dashboard
- ✓ KMIA + KMDW añadidos al scope curado
- ✓ Filtros anti-pérdidas (bias/streak block, mid-bin guard, divergence overnight, auto-cleanup)
- ✓ Tail-negation rule documentada (NO $[\leq X]$ con edge $\geq 40pp$ preferido sobre point-bets)
- ✓ Settle exclusivo NWS CLI (sin fallback Open-Meteo)
- ✓ Codex review 2026-06-18 — P0+P2 aplicados, P1/P3 pendientes
- ✓ Crypto predictor paralelo en :8001 (BTC quarterly)
- ✓ Migración a Raspberry Pi 4B (2026-06-19) — 100.83.162.24, auto-start, 24/7
- ✓ Dashboard agregado :8080 con /analysis, /btc-quarter, /ai
- ✓ AI agent (monitor 15min + briefing matutino) deployado 2026-06-21

Pendientes:

1. **Validación umbrales dificultad con datos reales.** Tras 2+ semanas, medir si días "fácil" tienen mejor Brier que "difícil" y recalibrar.
2. **Activación gradual isotónica** conforme acumulemos días.
3. **Cross-station reweight** (obs matutinas de estaciones cercanas).
4. **Histórico dificultad vs accuracy** — tabla/gráfica correlacionando score con Brier real.
5. **AI agent /ask chat** — endpoint de conversación con Sonnet.
6. **Post-mortem agent** — lecciones automáticas inyectadas al prompt.
7. **Codex review P1+P3** pendientes de aplicar.

Apéndice: métricas mencionadas

- **Brier score:** error cuadrático medio entre prob predicha y outcome (0/1). Menor = mejor.
- **Reliability diagram:** por bucket de prob predicha, qué fracción ocurrió. Si cae en diagonal, calibrado.
- **Kelly fraction:** fracción óptima del bankroll para maximizar crecimiento log esperado.
- **Edge en pp:** diferencia entre tu prob y la del mercado. Positivo = mercado subestima.
- **eff N:** miembros efectivos del ensemble tras reweight bayesiano. Bajo respecto a 31 = ensemble no capturó la realidad reciente.
- **Difficulty 0-100:** score combinado (spread + eff_n + clim + precip + regime). ≥ 75 muy difícil, ≥ 55 difícil, ≥ 30 normal.
- **Ruptura de régimen:** ≥ 2 horas con obs fuera de p1–p99 del ensemble. Fuerza difficulty=100, bloquea entries.
- **Conviction tier (AI agent):** high = edge $\geq 30pp$ + 3 señales convergen + difficulty ≤ 70 ; med = 15-30pp; low no se reporta.